4-2 Milestone Three: Enhancement Two: Algorithms and Data Structure

Andrew Wilson

Computer Science Department, Southern New Hampshire University

CS 499: Computer Science Capstone

Joseph Conlan

46111March 29, 2026

4-2 Milestone Three: Enhancement Two: Algorithms and Data Structure

In this artifact, I performed a large amount of refactoring and implementation of a complicated use case for combining records from several different repositories into a set of data structures for UI consumption.

I overhauled the navigation system to use modern type-safe Compose Navigation using the Serializable plug-in module to avoid manually building destination route strings that are vulnerable to route injection and generally brittle. This resulted in a much cleaner app navigation graph and navigation controller code. I split the app opening process into a landing "splash screen" that performs asynchronous checks to determine if the "first time app has been opened" onboarding screen, or "no data in database yet, let's set up your garden" screen, or the main Summary screen is chosen.

I added a new screen called Watering List View, which the user can access by tapping the "Last Watering" info card on the Summary screen. This screen consolidates and displays watering records for all active crops, grouped by plot. Displaying this information requires collecting data from three different repositories: the `CropsRepository`, `PlotsRepository`, and `WateringRecordsRepository`. Because the data in these repositories has been designed around normalized backend database tables, a fair amount of cross-referencing is necessary to put the data into a form usable by the Watering List View composable UI function. I used a series of data structures and algorithms to accomplish this, which can be seen in the files `GetNewWateringRecordsForAllActiveCropsUseCase.kt` and `WateringListViewModel.kt`. My method is to first get the list of all active crops from the CropsRepository, for which there is a method that retrieves this directly. I then use the groupBy method to group these crops by their plotId, resulting in a Map of (plotId → cropEntity). I then set up a list of deferred fetches of the

watering records for each crop from the WateringRecordsRepository using Kotlin coroutines. These are dispatched and collected together asynchronously into a flattened and mapped list such that the plotId is the key, and the value is all watering records for crops in that plot.

The WaterListViewModel retrieves this map of plotId → wateringRecords and asynchronously retrieves the plot names for each plotId and then transforms the pieces of data inside the watering records into a specific set of nested data structures that the WaterList composable UI function can use. This is finally rendered in the UI using loops to populate the list, grouped by plot, and displaying the last date and crop for each watering record.

AI Assistance Used

I used Gemini and gemma4, qwen3.5, and gpt-oss:20b, along with reading the Google Android documentation, to ask questions to fill out my understanding of the Kotlin corountine system, which mapping functions are available in Kotlin, and how to deal with Java and Kotlin differences with Instants and DateTime object representations. I did not directly incorporate any code samples from any GenAI output into my application.

References